COMP 3059 – Capstone Project I

Software Requirements Analysis and Design Assignment

This assignment is an overview to gather the software needs with requirements analysis and help to proceed with the design.

The requirements analysis helps to break down functional and non-functional requirements to a basic design view to provide a clear system development process framework. It involves various entities, including business, stakeholders and technology requirements.

The design is the activity following requirements specification and before programming. Software design usually involves problem solving and planning a software solution.

To work on this assignment, you could use the references and a sample template given below. The sample template can be customised to suit the nature of your project.

Reference Reading/Example to work on this Sprint document:
www.cse.msu.edu/~chengb/RE-491/Papers/SRSExample-webapp.doc

Source for this template:
www.tricity.wsu.edu/~mckinnon/cpts322/cpts322-srs-v1.doc

Table of Contents

1.0 Introduction.....3
1.1Purpose.....3
1.2 Scope.....3
2.0 System Overview.....3
2.1 Project Perspective.....3
2.2 System Context.....3
2.3 General Constraints.....3
2.4 Assumptions and Dependencies.....4
3.0 Functional Requirements 4
3.1 High Level Requirements
3.1.1 User Accounts & Profiles (HLR01, Priority: High) 4
3.1.2 External Game Data Integration (HLR02, Priority: High) 4
3.1.3 Game Logging & Backlog Categories (HLR03, Priority: High) 4
3.1.4 Ratings & Reviews (HLR04, Priority: High) 5
3.1.5 Backlog Management Tools (HLR05, Priority: Medium) 5
3.1.6 Social Features (Follow, Feed, Interactions) (HLR06, Priority: Medium) 5
3.1.7 Analytics & Gamification (HLR07, Priority: Low) 5
3.2 Use Cases 6
3.2.1 Register/Login 6
3.2.2 Search & View Game Details 6
3.2.3 Add Game to Log / Change Category 6
3.2.4 Write/Edit/Delete Review 6
3.2.5 Follow User & View Feed 6
3.3 Data Modelling and Analysis 7
3.4 Process Modelling 11
4.0 Non-Functional Requirements 13
4.1 Reliability & Availability 13
4.2 Security 13
4.3 Maintainability 13
4.4 Portability & Compatibility 14
4.5 Usability 14
5.0 Logical Database Requirements 14
6.0 Other Requirements 14

1.0 Introduction

1.1 Purpose

This document specifies the software requirements for GameBoxd – A Social Video Game Journal & Discovery Platform. It defines what the system must do for its stakeholders and users, independent of implementation details, and provides a basis for analysis, design, verification, and acceptance.

1.2 Scope

GameBoxd is a cross-platform web and mobile application that enables gamers to log titles, rate/review games, and discover content via a social activity feed. The system integrates with third-party game databases to retrieve authoritative metadata.

In scope: authentication and profiles, game logging, ratings/reviews, external API integration, social feed (follow/like/comment), basic analytics/gamification.

Out of scope (Capstone I): monetization, commerce, console/launcher integrations, offline tracking.

Primary objectives:

- Provide a **community-first** review and discovery experience.
- Enable **personal journaling** and backlog management.
- Use **trusted APIs** for accurate game data.
- Establish a **scalable design** for implementation in Capstone II.

2.0 System Overview

2.1 Project Perspective

The system is a new, self-contained product. It reuses no prior internal systems and relies on external data providers (RAWG/IGDB) and an external auth provider (Firebase).

2.2 System Context

Actors: End users (players), friends/community, external game data APIs, authentication provider (Firebase).

Context: Users authenticate, search/fetch game metadata from external APIs, create logs/reviews, and interact socially. The platform persists user-generated data and renders feeds on web and mobile clients.

2.3 General Constraints

- Academic schedule and milestones (Capstone I–II).
- External API rate limits and terms of use.
- Security and privacy requirements for user accounts and content.
- Cross-platform UI conventions (responsive web, iOS/Android).
- Hosting budget (student/free tiers).

2.4 Assumptions and Dependencies

Assumptions:

- Continuous availability of RAWG/IGDB endpoints.
- Reliable internet connectivity for users.
- Firebase Authentication supports required auth flows

Dependencies:

- RAWG/IGDB APIs for game metadata.
- Firebase Authentication for identity and session management.
- Cloud hosting/CI for deployment and availability.

3.0 Functional Requirements

3.1.1 User Accounts & Profiles (HLR01, Priority: High)

Provide secure registration/login and basic profiles to store user identity and preferences.

Inputs

- Registration data: email, password (and optional display name/avatar).
- Login credentials; profile edits (bio, avatar).

Processing

- Validate credentials via **Firebase Authentication**.
- Create/read/update user profile records in the application datastore.
- Maintain session tokens and sign-out.

Outputs

- Authenticated session.
- Persisted profile with view/edit capability.

3.1.2 External Game Data Integration (HLR02, Priority: High)

cataloging Fetch authoritative game metadata to ensure accurate and search.

Inputs

- Search queries, game IDs, pagination parameters.

Processing

- Call RAWG/IGDB endpoints; handle rate limits and errors.
- Normalize and cache essential fields (title, platforms, genres, release date, cover art).

Outputs

- Structured game objects for UI lists, detail pages, logging flows.

3.1.3 Game Logging & Backlog Categories (HLR03, Priority: High)

Allow users to classify games as *Playing, Completed, Wishlist, Backlog*.

Inputs

- User selections to add/move/remove a game in a category.

Processing

- Persist per-user game state; enforce one state per game per user.
- Support moves between states; record timestamps.

Outputs

- User's personal log views filtered by category; history of changes.

3.1.4 Ratings & Reviews (HLR04, Priority: High)

Enable user ratings/reviews and compute community insights.

Inputs

- Rating (e.g., 1–5 or 0.5 increments), review text, edits/deletions.

Processing

- Validate content (length, profanity filter baseline).
- Store review; update **aggregated rating** (e.g., average, count).
- Support edit/delete with audit trail (timestamped).

Outputs

- Review feed per game; user review history; community aggregates.

3.1.5 Backlog Management Tools (HLR05, Priority: Medium)

Provide reminders and prioritization suggestions to help reduce backlog.

Inputs

- User preferences, backlog items, optional target dates.

Processing

- Generate **non-intrusive reminders** (in-app; email optional).
- Simple prioritization (e.g., shortest length first, release recency).
- Allow snooze/disable per item.

Outputs

- Reminder notifications; prioritized backlog list.

3.1.6 Social Features (Follow, Feed, Interactions) (HLR06, Priority: Medium)

Support basic social graph and interactions to enable discovery.

Inputs

- Follow/unfollow actions; like/comment on reviews or log events.

Processing

- Persist follow relationships; build **activity feed** from followed users' actions.
- Record likes/comments; moderate simple abuse flags.

Outputs

- Personalized feed; engagement counts; comment threads.

3.1.7 Analytics & Gamification (HLR07, Priority: Low)

Motivate engagement via badges and personal stats.

Inputs

- User activity (games completed, reviews written, streaks).

Processing

- Compute stats (counts, categories played); evaluate badge rules.
- Update badge inventory.

Outputs

- Stats dashboard; badge gallery; optional share prompts.

3.2 Use Cases

3.2.1 Register / Login

Primary Actor: User
Preconditions: App installed or web reachable; user provides valid credentials.
Main Flow:

1. User submits email/password.
2. System authenticates with Firebase and creates/returns a session.
3. User lands on home/log view.
   Postconditions: Valid session; profile created if first time.
   Exceptions: Invalid password; network/API errors → show error/retry.

3.2.2 Search & View Game Details

Actor: User
Flow:

1. User searches a title.
2. System queries RAWG/IGDB; shows results.
3. User opens a game to view metadata, ratings, and reviews.
   Postconditions: None (read-only).
   Exceptions: API timeout → fallback message/limited cached data.

3.2.3 Add Game to Log / Change Category

Actor: User
Flow:

1. From a game page, user selects a category (Playing/Completed/etc.).
2. System persists state and timestamp.
3. Log view reflects the change immediately.
   Exceptions: Conflict/network error → revert and notify.

3.2.4 Write/Edit/Delete Review

Actor: User
Flow:

1. User writes a rating/review and submits.
2. System validates, stores, updates aggregates.
3. User may edit or delete; system updates accordingly.
   Exceptions: Validation failure → prompt correction.

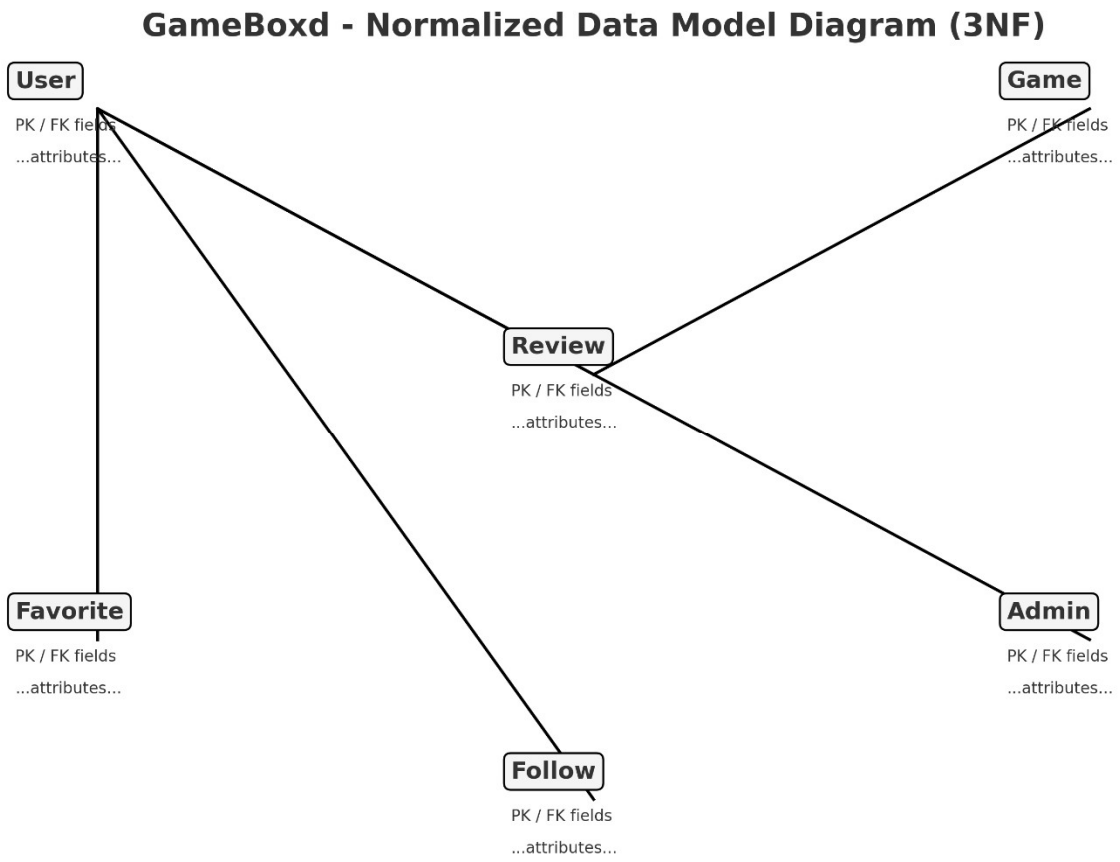
3.2.5 Follow User & View Feed

Actor: User
Flow:

1. User follows another user.
2. System adds relationship and populates feed with followed user's actions.
3. User likes/comments on items.
   Exceptions: Privacy settings/abuse flags → restrict visibility.

3.3 Data Modelling and Analysis

Normalized Data Model Diagram

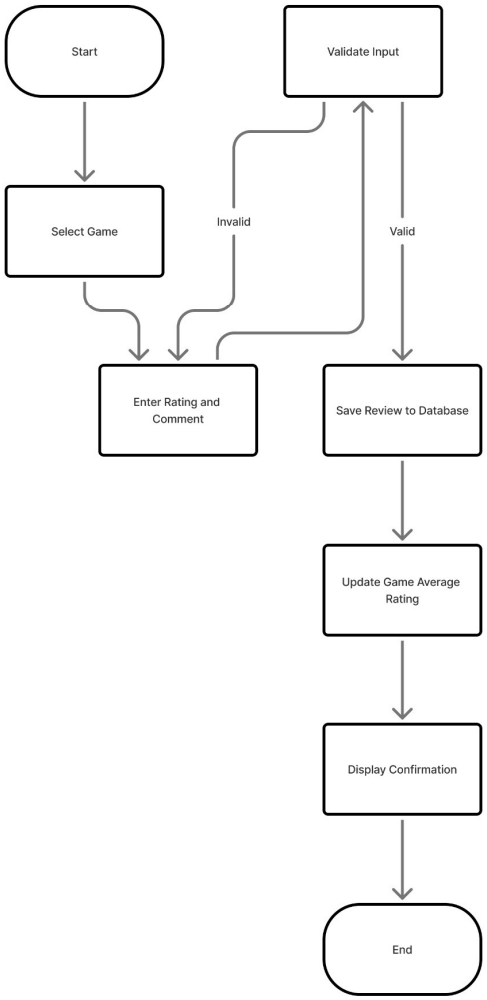


Entities and Relationships

User: Stores essential user information including username, email, and profile details. Relationships: one-to-many with Review, many-to-many with Game via Favorite, self-referencing many-to-many through Follow, and one-to-one with Admin.
Game: Represents game-related metadata such as title, genre, release date, and publisher. Relationships: one-to-many with Review, and many-to-many with User through Favorite.
Review: Serves as a junction entity connecting User and Game, containing rating, comment, and review date. Each review links a single user to a single game.
Favorite: Implements a many-to-many relationship between users and games, allowing users to bookmark games they enjoy or wish to play.
Follow: Enables social interaction, allowing one user to follow another. It's a self-referencing associative entity that tracks the follower–following relationship.
Admin: Represents elevated privileges within the system, linked directly to a User account through a one-to-one relationship. Admins can manage users and games.

Normalization Step	Objective	Implementation in GameBoxd
1NF (Atomicity)	Ensure each attribute holds indivisible data values.	All columns store single values (e.g., one email per user, one rating per review).
2NF (Partial Dependency Removal)	Eliminate dependencies on part of a composite key.	In Review, attributes depend on the full combination of user_id and game_id.
3NF (Transitive Dependency Removal)	Remove attributes that depend on non-key attributes.	Non-key data like user profile info is kept separate from review details.

Activity Diagrams

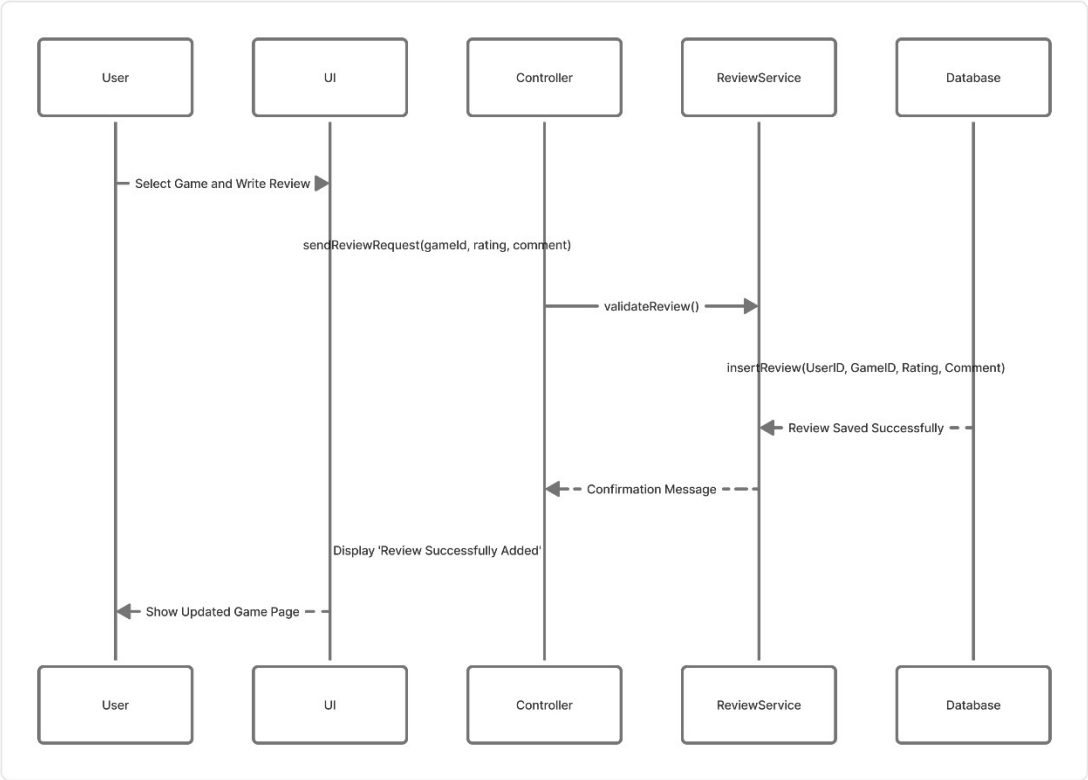


Analysis:
This diagram illustrates the logical flow of how a *GameBoxd* user posts a review:

1. The process begins when a **user selects a game** from the platform.
2. The user **enters a rating and comment**, which is then **validated** by the system.
3. If validation passes, the **review is saved** to the database; otherwise, the system prompts the user to correct inputs.
4. The platform then **updates the game's average rating** and **confirms** successful submission.

It visually represents the **sequential control logic** of this user interaction.

Sequence Diagrams



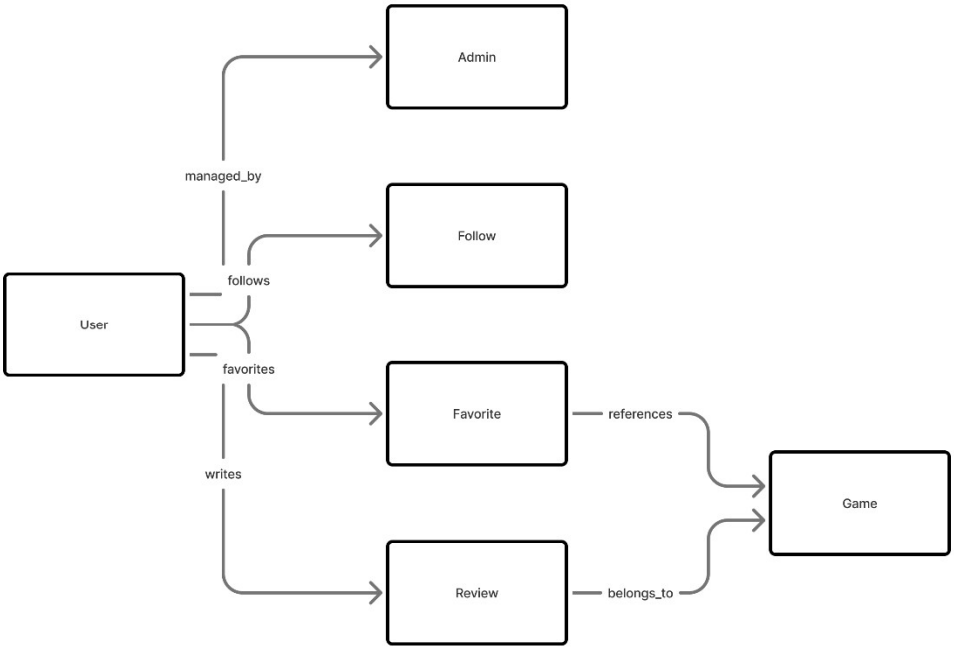
Analysis:
This sequence diagram demonstrates the **interaction flow** when a user submits a review on *GameBoxd*:

1. The **User** initiates the process through the UI by selecting a game and writing a review.
2. The **UI** forwards the data to the **Controller**, which manages communication with backend services.
3. The **Controller** calls the **ReviewService** to validate and process the review.
4. The **ReviewService** performs a database operation to **insert the review record**.

5. Upon success, the service sends a **confirmation message** back through the controller to the UI, which then notifies the user.

This diagram highlights the **synchronous request–response pattern** between layers, ensuring that reviews are validated and persisted reliably.

UML Class Diagram

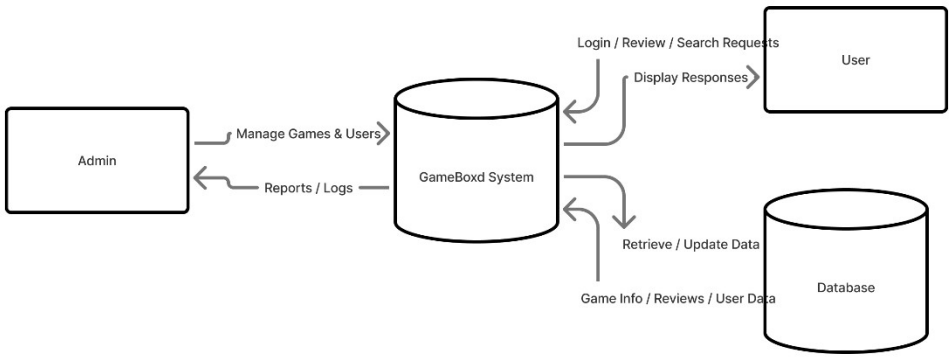


This UML Class Diagram models the structural backbone of the *GameBoxd* platform.

- The **User** class serves as the primary actor, connected to Review, Follow, and Favorite associations that represent user activities within the app.
- Each **Review** links a user to a specific **Game**, while **Favorite** captures a user's saved titles.
- The **Admin** entity supervises and manages the system's users and content.
- All classes are normalized and use one-to-many or many-to-many relationships to maintain modularity and reduce redundancy.

3.4 Process Modelling

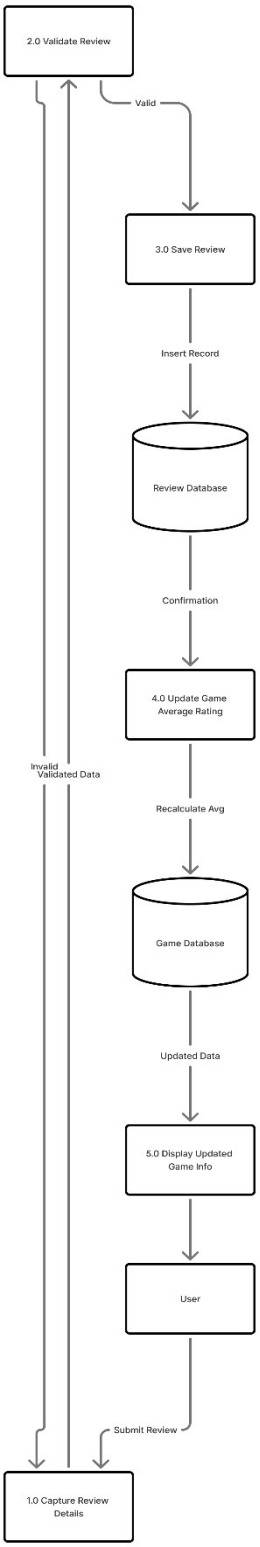
- Data Flow Diagram
  DFD Level 0 → flowchart (context-level data flow)



Analysis:
This diagram provides a **high-level overview** of the *GameBoxd* platform's external interactions and major data flows.

- The **User** communicates with the **GameBoxd System** for actions like login, searching, reviewing, and browsing.
- The **Admin** manages games, users, and reviews, interacting through dedicated interfaces.
- The **Database** serves as the central repository for user profiles, game metadata, and review information.
- Data flows clearly depict **input/output relationships**, showcasing GameBoxd as a **central processing unit** that mediates between human users and the underlying database.

DFD Level 1 → flowchart (review subsystem flow)



Analysis:
This diagram decomposes the *GameBoxd* system's **review management process** into detailed components:

1. The **User** submits a review via the UI, initiating the process.
2. The system **captures review details** and proceeds to **validate inputs** (rating range, text, game association).
3. If valid, data is **saved into the Review Database**; otherwise, the user is prompted to revise their input.
4. The system then **recalculates the game's average rating**, updating the **Game Database**.
5. Finally, the platform **displays the updated game information** back to the user interface.

This DFD illustrates the **internal logic and data transformations** within GameBoxd's review subsystem, ensuring both data integrity and responsive updates.

4.0 Non-Functional Requirements

4.1 Performance

- 95% of API-backed **search requests** shall return results within **1.5 seconds** under normal load.
- 95% of **log update** actions shall complete within **800 ms** (server acknowledgment).
- Feed loading (first screenful) shall render within **1.2 seconds** for cached items.

4.2 Reliability & Availability

- Service uptime target **≥ 99.0%** (excluding scheduled maintenance).
- Any single failure shall not corrupt persisted user data (use atomic writes/transactions where applicable).

4.3 Security

- Authentication via **Firebase Authentication**
- Passwords never stored by the application; rely on Firebase credential storage.
- Enforce **RBAC** minimum: authenticated user vs. anonymous; users can modify only their own content.
- Input validation and basic content moderation (length limits, profanity filter baseline).
- Protect API keys/secrets in server-side or secure config (never in client bundle).

4.4 Maintainability

- Codebase shall include automated **linting** and **unit tests** with ≥ **60%** coverage for core modules by Beta (Capstone II).
- CI pipeline shall run tests and block merges on failure.

4.5 Portability & Compatibility

- Responsive web UI (≥ 360px width) across latest Chrome/Edge/Safari/Firefox.
- Mobile builds shall support **Android 10+** and **iOS 15+**.
- External APIs shall be abstracted behind a service layer to enable provider changes with **≤ 3 person-days** of refactor for equivalent fields.

4.6 Usability

- First-time user can add a game to a log in **≤ 3 clicks/taps** from home.
- Key tasks (search, add to log, write review) shall be discoverable without tutorial (first-time success rate ≥ **80%** in informal hallway testing).

5.0 Logical Database Requirements

- **Entities** (logical): User, Profile, Game (external ID, normalized fields), UserGameState (userId, gameId, category, notes, timestamps), Review (userId, gameId, rating, text, timestamps), Follow (followerId, followeeId), Like, Comment, Badge, UserBadge
- **Keys & Integrity**:
  - Primary keys per entity; foreign keys between User→Review, User→UserGameState, User→Follow, Game→(Review, UserGameState).
  - Enforce one active UserGameState per (userId,gameId).
- **Retention**:
  - Soft-delete for reviews/comments; retain edit history timestamps.
- **Data Quality**:
  - Normalize and cache external metadata with source, sourceId, lastSyncedAt.
- **Scalability**:
  - Indexes on (userId,gameId) pairs; text index on game title for local search fallback.

6.0 Other Requirements

- **Legal/Compliance**: Respect API terms of service; attribute data sources where required. Provide minimal privacy notice (what data stored and why).

7.0 Approval

The signatures below indicate their approval of the contents of this document.

Project Role	Name	Signature	Date
Leader	Thinh Phan	TP	11/05/2025